

Number: P3125R4

Date: 2025-05-19

Audience: [Library Evolution](#)

Author: [Hana Dusíková](#)

- [Acknowledgement](#)
- [Revision history](#)
- [Introduction and motivation](#)
 - [Use cases](#)
 - [Safety](#)
 - [Unsafe operations](#)
 - [Examples](#)
 - [HAMT early leaves](#)
 - [Smart \(non-\)owning pointer](#)
 - [LLVM's L-value](#)
- [Implementation experience](#)
 - [Implementation in the library](#)
 - [Accessing raw tagged pointers](#)
 - [Implementation in the compiler](#)
 - [Compiler builtins](#)
 - [Constant evaluation](#)
 - [Pointer provenance and optimization](#)
 - [Alternative constexpr compatible implementation](#)
- [Design](#)
 - [Preconditions and eligibility of constructors](#)
 - [Representation of tag value](#)
 - [Tuple protocol](#)
- [Things it's not doing and why](#)
- [Impact on existing code](#)
- [Proposed changes to wording](#)
 - [Feature test macro](#)

constexpr pointer tagging

This paper proposes a new library non-owning pointer and value pair which has ability to store small amount of information in a *tag* in alignment low-bits. This functionality is also usable in constexpr environment. It's meant to be a building tool to build more advanced data-structures and also last requirement for `atomic<std::shared_ptr<T>>` to be constexpr.

Acknowledgement

I want to thank to everyone who helped me by reviewing this paper. I want to thank especially Tomasz Kamiński who made the wording actually make sense.

Revision history

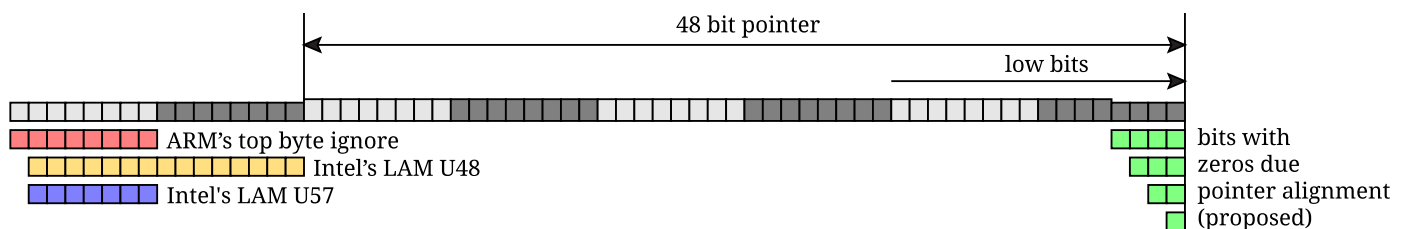
- [R3](#) → [R4](#): changes requested by LEWG (removed `nullptr_t` constructor, [added support for void *](#), [first template argument is now pointer and not pointee](#), added [pointer_tag_trait](#) to inspect available bits inside pointers, added [tuple_size](#), [tuple_element](#), and [get<N>\(pointer_tag_ptr\)](#))
- [R2](#) → [R3](#): polishing design and [added wording](#)
- [R1](#) → [R2](#): simplifying design by removing schemas and the heavy interface, massive simplification as discussed in SG1 (`pointer_tag_pair`, `required sizeof(pointer_tag_pair) == sizeof(T*)`)
- [R0](#) → [R1](#): proposing and explaining design based on existing implementation

Introduction and motivation

Pointer tagging is widely known and used technique ([Glasgow Haskell Compiler](#), LLVM's [PointerIntPair](#), [PointerUnion](#), [CPython's garbage collector](#), [Objective C](#) / Swift, Chrome's [V8 JavaScript engine](#), [GAP](#), [OCaml](#), [PBRT](#)). All major CPU vendors provides mechanism for pointer tagging (Intel's LAM [linear address masking](#), AMD's [Upper Address Ignore](#), ARM's TBI [top byte ignore](#) and MTE [memory tagging extension](#)). All widely used 64 bit platforms are [not using more than 48 or 49 bits of the pointers](#).

This functionality widely supported can't be expressed in a standard conforming way.

[Rust](#), [Dlang](#), or [Zig](#) has an interface for pointer tagging. This is demonstrating demand for the feature and C++ should have it too and it should also work in `constexpr`.



Generally programmer can consider low-bits used for alignment to be safely used for storing an information. Upper bits are available on different platforms under different conditions (runtime processor setting, CPU generation, ...). This proposal only proposes interface to accessing low bits for storing // reading a tag value, not observing bits of a pointer.

This proposal doesn't propose accessing any other bits other than low-bits which are known to be zero due alignment. [SG1 doesn't want](#) to standardize access to high-bits as it's considered dangerous and non-portable. And can limit future development of security features in OS and CPUs for which high-bits are used.

Use cases

There are three basic use-cases of pointer tagging:

- marking pointer with an information (in allocators, used as a tiny refcount, or marking source of the pointer)
- pointee polymorphism (usually in data structures, eg. in trees: next node can be an internal node or a leaf)
- in-place polymorphism (similarly as variant, some bits stores an information about rest of payload, it can be a pointer, a number, a float, a small string, ...)

This paper aims to solve only first two use-cases.

Safety

Pointer tagging can be currently implemented in C++ only with `reinterpret_cast` and bit manipulating with shifts / `bitand` / `bitor` and this approach is prone to be unsafe and hard to debug as it's not expressed clearly in code what is the right intention, so compiler can't even diagnose incompatibility between encode/decode. By giving a name to this tool it allows programmer to express intent clearly and compiler to optimize and diagnose problems properly.

Preconditions and mandates of proposed `std::pointer_tag_pair` makes it unlikely to use it unsafely as potentially dangerous operations (`std::pointer_tag_pair<Pointee, Tag, Bits>::from_tagged(pointer)` and `std::pointer_tag_pair<Pointee, Tag, Bits>::template from_overaligned<PromisedAlignment>(pointer)`) are verbose and visible.

Unsafe operations

Two mentioned functions are provided so user can explicitly provide over-aligned pointer which would otherwise won't be compatible with number of requested bits or interact with existing functionality for pointer tagging (which will allow gradual adoption of the feature and replace old code).

Examples

HAMT early leaves

Following example is a recursive search implementation for a [HAMT \(hash-array-mapped-trie\)](#) data structure. Where a tag value indicates leaf node.

```

// requesting only 1 bit of information
using hamt_node_pointer = std::pointer_tag_pair<const void, bool, 1>;
static_assert(sizeof(hamt_node_pointer) == sizeof(void *));

constexpr const T * find_value_in_hamt(hamt_node_pointer tptr, uint32_t hash) {
    if (tptr == nullptr) // checks only pointer part
        return nullptr;

    if (tptr.tag()) // we found leaf node, tag is boolean as specified
        return *static_cast<const T *>(tptr.pointer());

    const auto * node = static_cast<const internal_node *>(tptr.pointer());
    const auto next_node = node[hash & 0b1111u];

    return find_value_in_hamt(next_node, hash >> 4); // recursive descend
}

```

Smart (non-)owning pointer

This example shows `maybe_owning_ptr` type which can be both a reference or an owner:

```

template <typename T> class maybe_owning_ptr {
    enum class ownership: unsigned {
        reference,
        owning,
    };

    std::pointer_tag_pair<T, ownership, 1> _ptr;
public:
    constexpr maybe_owning_ptr(T* && pointer) noexcept: _ptr{pointer, ownership::own
ing} { }
    constexpr maybe_owning_ptr(T & ref) noexcept: _ptr{&ref, ownership::reference}
{ }

    constexpr decltype(auto) operator*() const noexcept {
        return *_ptr.pointer();
    }

    constexpr T * operator->() const noexcept {
        return _ptr.pointer();
    }

    constexpr ~maybe_owning_ptr() noexcept {
        if (_ptr.tag() == ownership::owning) {
            delete _ptr.pointer();
        }
    }
};

static_assert(sizeof(maybe_owning_ptr<int>) == sizeof(int *));

```

LLVM's L-value

Following code is simplification of LLVM's representation of [pointer/reference type](#) which is implemented with LLVM's `PointerIntPair` and is inside constant evaluator. This type holds subnodes of different kind: pointer to local/static variable, dynamic allocation, type info pointer, result of temporary.

```

struct LValueBase {
    using PtrTy = llvm::PointerUnion<const ValueDecl *,
                                    const Expr *,
                                    TypeInfoLValue,
                                    DynamicAllocLValue>;

    struct PathEntry {
        uint64_t value;
    };

    using PathTy = std::vector<PathEntry>;

    PtrTy Location;
    PathEntry SubObjectPath;
};

APValue DereferencePointer(EvalInfo & Context, const LValueBase & Base) {
    auto & object = Context.Visit(Base.Location);
    return object.NavigateToSubobject(Base.SubObjectPath);
}

```

Implementation experience

Old version of this proposal has been implemented within libc++ & clang and it is accessible on [github](#) and [compiler explorer](#). This functionality can't be implemented as a pure library (reinterpret_cast is not allowed during constant evaluation) and needs compiler support in some form.

Implementation in the library

Library is providing a special pair-like type containing pointer and small tag type, and user requested number of bits. The number of bits is by default deduced by default alignment of requested pointee type. Requesting more bits than alignment will disable normal constructor, and force user to use ::from_overaligned function as overalignment is not part of C++'s type system.

In terms of library design there is nothing surprising, and it's pretty straightforward wrapper which encode and decode tagged pointer on its boundaries and provides basic pointer functionality.

Accessing raw tagged pointers

The pointer int pair has support to access *raw* tagged pointer. Which is a pointer which can't be dereferenced or otherwise manipulated with, it's an opaque value usable only with legacy tagging interface or it can be used to construct `pointer_tag_pair` back from it. Existence of this interface allows ability to store such pointers in existing interfaces (atomic, other smart pointers). Question is if it should be `uintptr_t` or `void*`. I prefer `void*` as roundtripping thru an integer loses information about provenance and can disable some optimization.

Implementation in the compiler

The implementation is providing builtins to manipulating raw pointers and isn't meant to be used by end-users, only to allow this library functionality.

Compiler builtins

Implementation needs to manipulate pointers without casting them to integers and back. To do so the provided set of builtins is designed to store/load a value (with size of few bits) into/from unimportant/unused bits of a pointer without observing actual pointer representation.

Constant evaluation

With these builtins it's trivial to implement semantically identical behaviour for the constant evaluation. In case of my implementation, pointers in clang are not represented as addresses but as symbols (original AST variable, allocation, static object + path to subobject, its provenance) and there is no address to manipulate. Actual tag value in such "pointer" is then stored in a metadata of the pointer itself and builtins only provide access to it. Technically such storage can provide more bits than pointer "size", but there are internal checks which make sure it allows only bits which would be accessible in runtime based on alignment of individual pointer.

Any attempt to deference or otherwise manipulate such pointer, which would be unsafe in runtime, is detected and reported by the interpreter. Only the provided builtins can recover original pointer and tag value.

Pointer provenance and optimization

Easiest way to implement builtins for pointer tagging is to do the same thing `reinterpret_cast` is doing, which was my first implementation approach. But this approach leads to losing pointer's provenance and compiler losing information which otherwise should be accessible for optimizer to use.

For unmasking there is already `ptr.mask` LLVM's builtin, but there is no similar intrinsic to do the tagging. Hence the builtins needs to interact with backend and be implemented with a low level backend intrinsic to do the right thing. This shows how actually unimplementable pointer tagging is in existing language.

Alternative constexpr compatible implementation

Alternative way to implement `constexpr` support (for compiler which don't have heavy pointer representation in their interprets) is inserting a hidden intermediate object holding the metadata and pointer to original object. This allows exactly same semantic as the metadata approach, and can be completely implemented in library using `if constexpr`, but it will need allocation during constant evaluation.

Design

The `std::pointer_int_pair` is simple pair-like template providing only necessary interface and is not meant to provide heavy interface as it's preferable to not hide pointer tagging / untagging from users. This is mean to be a low-level facility. Main requirement on the type is it must be always same size as stored pointer and not more.

This chapter was partially removed because it was too similar to wording, you can find it in [previous version of paper](#).

Preconditions and eligibility of constructors

Constructor taking a pointer and tag value is only available if alignment of the pointee type is enough to store requested number of bits.

Both the constructor and `::from_overaligned` function have preconditions checking if pointer is aligned enough as expected (in the constructor) or promised overaligned (in the `::from_overaligned` function). In addition to this there is a precondition to make sure value of tag type is representible with `RequestedBits`.

Representation of tag value

Tag value can only be unsigned integral type or an enum type with underlying unsigned integral type. The value is converted to the underlying or kept original unsigned integral type and then it is bit masked with mask based on *BitsRequested* bits (`(1u << BitsRequested) - 1u`).

It's precondition failure for the value after this conversion different than original value. An attempt was made to support signed type, but unfortunately storing unrepresentable value into a bitfield is implementation specific. And this can be added later as extension to current design.

Tuple protocol

`pointer_tag_pair` supports being destructured, but it doesn't model *tuple-like* (as it would open whole can of worms, as told by STL). But following code should work:

```
auto [ptr, tag] = a_pointer_tag_pair;
```

Things it's not doing and why

- *modeling pointer* — this is not a pointer type, access to the pointer should be explicitly visible,
- *convertible to bool* — it's not sure if it means `.pointer() == nullptr` or also `.tag() == 0`,
- *manage lifetime* — this is not a owning pointer, it's a tool to build one,
- *using other than non-alignment bits* — these are not portable and are subject of being enabled/disabled as an OS setting, this would create at best ABI problems,
- *changing size based on bits requested* — intention of this type to be same size of pointer, not be generic pair of pointer and any value,
- *supporting function pointers* — these doesn't need to be real pointer, but handlers, and can even have bigger size than normal pointer.
- *supporting signed tag types* — it's tricky and using bitfield is implementation specific.

Impact on existing code

None, this is purely an API extension. It allows to express semantic clearly for a compiler instead of using an unsafe `reinterpret_cast` based techniques. Integral part of the proposed design is ability to interact with such existing code and migrate away from it.

Proposed changes to wording

20	Memory management library	[mem]
----	---------------------------	-------

20.1	General	[mem.general]
------	---------	---------------

1 This Clause describes components for memory management.

- 2 The following subclauses describe general memory management facilities, smart pointers, **pointer tagging**, memory resources, and scoped allocators, as summarized in Table 47.

Table 47 — Memory management library summary [\[tab:mem.summary\]](#)

	Subclause	Header
[memory]	Memory	<code><cstdlib></code> , <code><memory></code>
[smartptr]	Smart pointers	<code><memory></code>
[ptrtag]	Pointer tagging	<code><memory></code>
[mem.res]	Memory resources	<code><memory_resource></code>
[allocator.adaptor]	Scoped allocators	<code><scoped_allocator></code>

20.2 Memory [memory]

20.2.2 Header `<memory>` synopsis [memory.syn]

- 1 The header `<memory>` defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, destroy objects, and construct objects in uninitialized memory buffers ([\[pointer.traits\]](#)–[\[specialized.addressof\]](#) and [\[specialized.algorithms\]](#)). The header also defines class templates `pointer_tag_traits` and `pointer_tag_pair` to support storing additional information in unused bits of pointers. Selection of the bits is implementation specific and the size of the pointer wrapping object is the same as the size of the original pointer. The header also defines the templates `unique_ptr`, `shared_ptr`, `weak_ptr`, `out_ptr_t`, `inout_ptr_t`, and various function templates that operate on objects of these types ([\[smartptr\]](#)).

```
namespace ranges {
    template<destructible T>
        constexpr void destroy_at(T* location) noexcept;
    // freestanding

    template<nothrow-input-iterator I, nothrow-sentinel-for<I> S>
        requires destructible<iter_value_t<I>>
        constexpr I destroy(I first, S last) noexcept;
    // freestanding

    template<nothrow-input-range R>
        requires destructible<range_value_t<R>>
        constexpr borrowed_iterator_t<R> destroy(R&& r) noexcept;
    // freestanding

    template<nothrow-input-iterator I>
```

```

        requires destructible<iter_value_t<I>>
        constexpr I destroy_n(I first, iter_difference_t<I> n) noexcept
pt;
        //freestanding
    }

```

```

// [ptrtag], pointer tagging
template<class Ptr>
    struct pointer_tag_traits; //freestanding
template<class Ptr, class TagT,
    unsigned BitsRequested = pointer_tag_traits<Ptr>::bits_available<>>
    class pointer_tag_pair; //freestanding

template<class Ptr, class TagT, unsigned BitsRequested>
    struct tuple_size<pointer_tag_pair<Ptr, TagT, BitsRequested>>
        : integral_constant<size_t, 2> { };
template<class Ptr, class TagT, unsigned BitsRequested>
    struct tuple_size<const pointer_tag_pair<Ptr, TagT, BitsRequested>>
        : integral_constant<size_t, 2> { };

template<class Ptr, class TagT, unsigned BitsRequested>
    struct tuple_element<0, pointer_tag_pair<Ptr, TagT, BitsRequested>>
{
    using type = Ptr;
};
template<class Ptr, class TagT, unsigned BitsRequested>
    struct tuple_element<1, pointer_tag_pair<Ptr, TagT, BitsRequested>>
{
    using type = TagT;
};

template<class Ptr, class TagT, unsigned BitsRequested>
    struct tuple_element<0, const pointer_tag_pair<Ptr, TagT, BitsRequested>>
{
    using type = Ptr;
};
template<class Ptr, class TagT, unsigned BitsRequested>
    struct tuple_element<1, const pointer_tag_pair<Ptr, TagT, BitsRequested>>
{
    using type = TagT;
};

```

```

// [unique.ptr], class template unique_ptr
template<class T> struct default_delete;
//freestanding
template<class T> struct default_delete<T[]>;

```

```
// freestanding
    template<class T, class D = default_delete<T>> class unique_ptr;
// freestanding
    template<class T, class D> class unique_ptr<T[], D>;
// freestanding
```

20.? Pointer tagging [ptrtag]

20?.1 Class template pointer_tag_traits [ptrtag.traits]

20?.1.1 General [ptrtag.traits.general]

The class template `pointer_tag_traits` provides the interface for accessing information about the free bits in pointer values that are usable in the `pointer_tag_pair` class template [ptrtag.pair].

```
namespace std {
    template <typename T> struct pointer_tag_traits; // see below

    template <typename Pointee>
    requires (std::is_object_v<Pointee>)
    struct pointer_tag_traits<Pointee *> {
        template <unsigned Alignment = alignof(Pointee)>
        requires has_single_bit(Alignment)
        static constexpr bits_available = see below;
    };

    template <> struct pointer_tag_traits<void *> {
        template <unsigned Alignment = 1>
        requires has_single_bit(Alignment)
        static constexpr bits_available = see below;
    };

    template <typename Pointee> struct pointer_tag_traits<const Pointee *>:
        pointer_tag_traits<Pointee *> { };
};
```

20?.1.3 pointer_tag_traits members [ptrtag.traits.members]

```
template <unsigned Alignment = alignof(Pointee)>
    requires has_single_bit(Alignment)
    static constexpr bits_available = see below;
```

```
// pointer_tag_traits<void *> specialization
template <unsigned Alignment = 1>
    requires has_single_bit(Alignment)
    static constexpr bits_available = see below;
```

Implementation specific number of unused bits in the `Ptr *` and `void *` pointer representation.

Note: on reasonable platforms this value is `count_zero(Alignment)`.

20.7.2 Class template `pointer_tag_pair` [ptrtag.pair]

20.7.2.1 General [ptrtag.pair.general]

The class template `pointer_tag_pair` provides a type to store an object pointer together with a tag value.

```
namespace std {
    template <typename Ptr, typename TagT,
        unsigned BitsRequested = pointer_tag_traits<Ptr>::bits_available<>>
    class pointer_tag_pair {    // freestanding
    public:
        using pointer_type      = Ptr;
        using tagged_pointer_type = conditional_t<is_const_v<pointer>, const void *, void *>;
        using tag_type          = TagT;
        static constexpr unsigned bits_requested = BitsRequested;

        // Constructors and assignment
        constexpr pointer_tag_pair() noexcept;

        template <convertible_to<pointer_type> P>
            requires((pointer_tag_traits<P>::bits_available<>) >= bits_requested)
            constexpr pointer_tag_pair(P p, tag_type t);

        // Special construction helpers
        template <unsigned PromisedAlignment, convertible_to<pointer_type> P>
            requires((pointer_tag_traits<P>::bits_available<PromisedAlignment>) >= bits_requested)
            static constexpr pointer_tag_pair from_overaligned(P p, tag_type t);
```

```

    static pointer_tag_pair from_tagged(tagged_pointer_type p) noexcept; // note:
not constexpr

    // Accessors
    tagged_pointer_type tagged_pointer() const noexcept; // note: not constexpr
    constexpr pointer_type pointer() const noexcept;
    constexpr tag_type tag() const noexcept;

    // Swap
    constexpr void swap(pointer_tag_pair& o) noexcept;

    // Comparisons
    friend constexpr see-below operator<=>(pointer_tag_pair lhs, pointer_tag_pair
rhs) noexcept;
    friend bool operator==(pointer_tag_pair, pointer_tag_pair) = default;

};
}

```

An object of class `pointer_tag_pair<Ptr, TagT, BitsRequested>` represents a pair of pointer value `ptr` of type `PtrT` and tag value `tag` of type `TagT`.

Each specialization `PT` of `pointer_tag_pair` is trivially copyable type that models copyable such that `sizeof(PT)` is equal `sizeof(Ptr)`

Mandates:

- `is_same_v<remove_cvref_t<Ptr>, Ptr> && is_same_v<remove_cvref_t<TagT>, TagT>` is true
- `is_pointer_v<Ptr> && !is_function_v<remove_pointer_t<Ptr>>` is true
- `is_unsigned_t<UT>` is true, where `UT` is `underlying_type_t<TagT>` if `TagT` is enumeration type, and `TagT` otherwise
- `sizeof(TagT) <= sizeof(void*)`.

20.2.2.2 Constructors and assignment [ptrtag.pair.cnstrassgnmt]

```
constexpr pointer_tag_pair() noexcept;
```

Effects: Value-initializes `ptr` and `tag`.

```

template <convertible_to<pointer_type> P>
    requires((pointer_tag_traits<P>::bits_available<>) >= bits_requested)
    constexpr pointer_tag_pair(P p, tag_type t);

```

Precondition: `bit_width(t) <= bits_requested` is true.

Effects: Initializes `ptr` with `p` and `tag` with `t`.

Throws: Nothing.

20.2.2.3 Support for overaligned pointers [`ptrtag.pair.overalign`]

```
template <unsigned PromisedAlignment, convertible_to<pointer_type> P>
    requires((pointer_tag_traits<P>::bits_available<PromisedAlignment>) >= bits_
requested)
    static constexpr pointer_tag_pair from_overaligned(P p, tag_type t);
```

Precondition:

- `is_sufficiently_aligned<PromisedAlignment>(p)` is true.
- `bit_width(t) <= bits_requested` is true.

Effects: Initializes `ptr` with `p` and `tag` with `t`.

Throws: Nothing.

20.2.2.4 Tagged pointer operations [`ptrtag.pair.tagops`]

```
tagged_pointer_type tagged_pointer() const noexcept; // note: not constexpr
```

Returns: An unspecified value `tp` of `tagged_pointer_type` type such that for any specialization `DP` of `pointer_tag_pair` and alignment value `A` for which:

- `pointer_tag_traits<void*>::bits_available<A> >= DP::bits_requested` is true,
- `is_sufficiently_aligned<A>(ptr)` is true, and
- `bit_width(tag) <= DP::bits_requested` is true,

`DP::from_tagged(tp)` produces object `dp` such that `reinterpret_cast<Ptr>(dp.pointer()) == ptr` is true and `static_cast<TagT>(dp.tag()) == tag` is true.

```
static pointer_tag_pair from_tagged(tagged_pointer_type p) noexcept; // note: not
constexpr
```

Returns: An object `dp` of type `pointer_tag_type`, such that:

- `reinterpret_cast<Ptr>(sp.pointer()) == dp.pointer()` is true and `static_cast<TagT>(sp.tag()) == dp.tag()` is true, if `p` is equal to `sp.tagged_pointer()` for some object `sp` of type that is specialization of `pointer_tag_type`, such that for some alignment value `A`:
 - `pointer_tag_traits<void*>::bits_available<A> >= bits_requested` is true,
 - `is_sufficiently_aligned<A>(sp.pointer())` is true, and
 - `bit_width(sp.tag()) <= bits_requested` is true,
- otherwise, values of `dp.pointer()` and `dp.tag()` are unspecified.

20.2.2.5 Accessors [`ptrtag.pair.accessors`]

```
constexpr pointer_type pointer() const noexcept;
```

Returns: `ptr`.

```
constexpr tag_type tag() const noexcept;
```

Returns: `tag`.

20.2.2.6 Swap [`ptrtag.pair.swap`]

```
constexpr void swap(pointer_tag_pair& o) noexcept;
```

Effects: Exchanges the values of `*this` and `o`.

20.2.2.7 Comparison [`ptrtag.pair.comparison`]

```
friend constexpr auto operator<=>(pointer_tag_pair lhs, pointer_tag_pair rhs) noexcept;
```

Returns: `tuple(lhs.pointer(), lhs.tag()) <=> tuple(rhs.pointer(), rhs.tag())`.

```
friend constexpr bool operator==(pointer_tag_pair lhs, pointer_tag_pair rhs) noexcept;
```

Returns: `lhs.pointer() == rhs.pointer() && lhs.tag() == rhs.tag()`.

20.2.8 Tuple interface [ptrtag.pair.get]

```
template<class Ptr, class TagT, unsigned BitsRequested>
constexpr tuple_element_t<I, pointer_tag_pair<Ptr, TagT, BitsRequested>>
    get(pointer_tag_pair<Ptr, TagT, BitsRequested> p) noexcept;
```

Mandates: $I < 2$

Returns:

- `p.pointer()` if I is equal to zero,
- `p.tag()` otherwise.

Feature test macro

17.3.2 Header <version> synopsis [version.syn]

```
#define __cpp_lib_pointer_tag_pair 2025??L // freestanding, also in <memory>
```